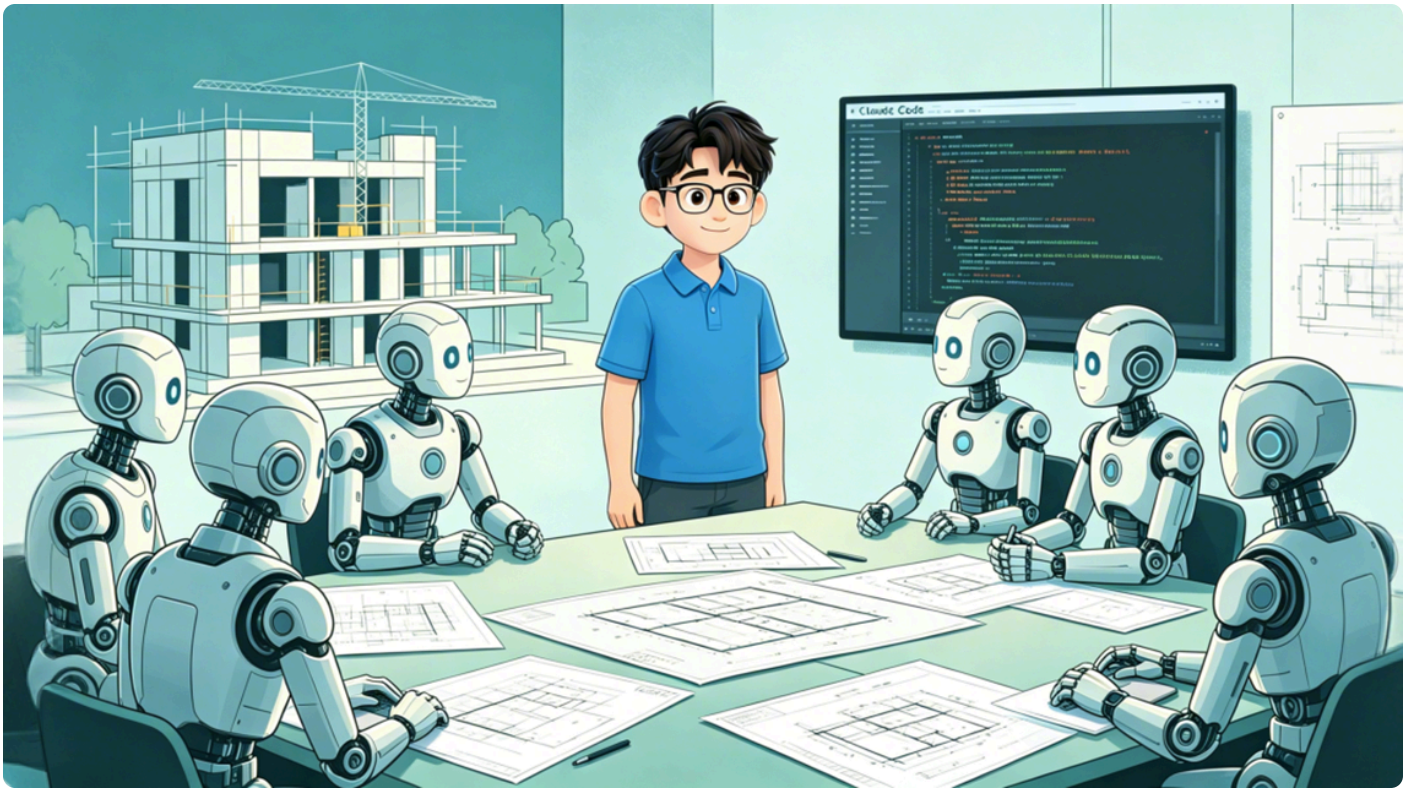
01 | 你是架构师，Claude Code 是你的工程团队

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

从这一讲开始，我们正式进入课程的认知篇。在动手写任何代码之前，我想先和你聊清楚一件事：你和 Claude Code 到底应该是什么关系？

这门课我们会围绕用 Claude Code 从零构建一个叫 Hify 的 AI Agent 开发平台来展开。但在动手之前，有一个比“怎么做”更重要的问题要先想清楚——**产品需求、架构设计、测试回归、部署运维，这些事情谁来做**？

以往这些可能已经由你的产品经理、架构师、测试团队、运维团队完成了。但当你只有 Claude Code 的时候，你就是所有事情的负责人。你需要想清楚这些事情，然后让 Claude Code 协助你完成。具体执行可能是 Claude Code 完成 90%，你完成 10%。

所以这一讲主要想讲清楚一个认知框架：你要先转变角色，才能用好工具。不要把 Claude Code 当作只会写代码的工具。它可以是架构师、产品经理、测试专家、运维专家

——关键是看你怎么激活它。这也是这门课在教的事情：怎么激活。

用 Claude Code 做项目，瓶颈不在 AI，在你。

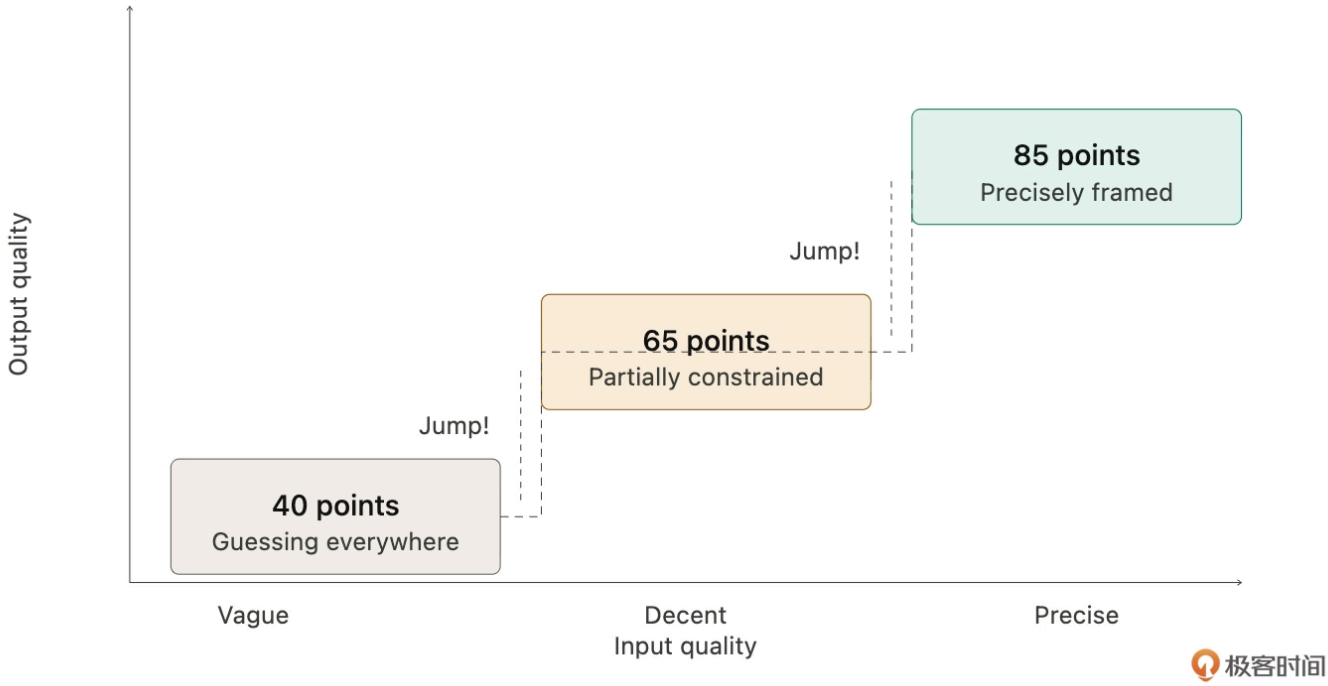
我会从三个方面展开：为什么同样的工具不同人用效果差那么多，你和 Claude Code 各自该干什么，以及拿到它的输出之后怎么判断能不能用。

为什么同样的工具，你用和别人用差这么多

开篇词里我讲过第一天踩坑的经历——直接让 Claude Code 设计架构，拿到一个过度设计的微服务方案。问题不在 AI，在于我没给它足够的约束和上下文。后来我花一个晚上想清楚产品边界、架构约束、技术规范，写成 CLAUDE.md 喂给它，第二天输出质量完全不同。

这里我想展开讲一个更本质的发现：**输入质量和输出质量之间不是线性关系，是阶梯式的**。

你给 AI 的输入从 60 分提升到 80 分，输出不是从 60 涨到 80，而是可能从 40 直接跳到 85。因为当你的描述模糊时，AI 需要“猜”——猜你要什么架构、猜你的设计意图、猜你的质量标准。每猜一个就可能猜错，错误还会叠加。但当你给的上下文足够清晰时，它不用猜了，输出质量会有质的变化。



X 轴是输入质量，Y 轴是输出质量

所以第一个要建立的认知是：你想得越清楚，它做得越准确；你越模糊，它越跑偏。瓶颈不在 AI 的能力上，在你的思考质量上。

那接下来的问题自然就是：我该想清楚哪些事情？我和 Claude Code 之间怎么分工？

你做决策，它做执行

我现在和 Claude Code 的协作方式，总结下来就一句话：我是架构师，它是我的工程团队。

这不是比喻。想想一个架构师每天干什么：

1. 他不写具体的业务代码，但他决定了系统长什么样。
2. 他定方向：做什么、不做什么。
3. 他做取舍：遇到矛盾选 A 还是选 B。
4. 他立标准：代码怎么写、接口怎么定、出了错怎么处理。
5. 他验收成果：团队交付的东西是不是他要的。

工程团队在干什么？在按标准高效执行。

你给 Claude Code 下达的每一条指令，本质上就是架构师在给团队派任务。我拿 Hify 里的一个真实场景举例，下面是两种方式：

第一种方式

问题是：“帮我用 Spring Boot 实现一个模型提供商管理的 CRUD 接口。”

Claude Code 的输出是：

接口设计

方法	路径	说明
`POST`	`/api/providers`	创建提供商
`GET`	`/api/providers?page=&size=&enabled=&type=`	分页查询（支持按状态/类型过滤）
`GET`	`/api/providers/{id}`	查询单个
`PUT`	`/api/providers/{id}`	全量更新
`PATCH`	`/api/providers/{id}/status`	启用/禁用切换
`DELETE`	`/api/providers/{id}`	删除

分层结构

```
controller/   → ProviderController        处理 HTTP 请求、参数校验
service/      → ProviderService            业务逻辑、名称唯一性校验
repository/   → ModelProviderRepository    JPA 数据访问
entity/       → ModelProvider              JPA 实体
dto/          → ProviderRequest            入参（带 @Valid 校验注解）
              → ProviderResponse           出参（屏蔽 apiKey 原文）
enums/        → ProviderType               提供商类型枚举
common/       → ApiResponse<T>             统一响应结构
              → PageResult<T>              分页响应包装
              → GlobalExceptionHandler     全局异常处理
```

统一响应结构

```
{
  "code": 200,
  "message": "success",
  "data": { ... }
}
```

错误示例：

```
{ "code": 404, "message": "提供商不存在" }
{ "code": 400, "message": "名称不能为空; Base URL 不能为空" }
```

第二种方式

问题是：“按照 CLAUDE.md 中的接口规范，实现模型提供商的 CRUD 接口，使用 MyBatis-Plus，错误码按规范定义，连通性测试的异常区分网络超时（2001）、认证失败（2002）、模型不存在（2003）三种情况。不需要工厂模式，用最简单的方式实现。”

Claude Code 的输出是：

三、错误码规范

```
// ErrorCode.java
public enum ErrorCode {
    // 通用
    SUCCESS(0, "success"),
    PARAM_ERROR(1001, "参数错误"),
    NOT_FOUND(1002, "资源不存在"),
    NAME_DUPLICATE(1003, "名称已存在"),

    // 连通性测试
    CONNECT_TIMEOUT(2001, "网络超时，无法连接到服务"),
    AUTH_FAILED(2002, "认证失败，API Key 无效或已过期"),
    MODEL_NOT_FOUND(2003, "模型不存在");
}
```

四、接口清单

方法	路径	说明
POST	`/api/model-providers`	创建提供商
GET	`/api/model-providers`	分页列表（`?page=1&size=10&type=OPENAI`）
GET	`/api/model-providers/{id}`	获取单个
PUT	`/api/model-providers/{id}`	更新
DELETE	`/api/model-providers/{id}`	删除（逻辑删除）
POST	`/api/model-providers/{id}/test`	连通性测试

五、连通性测试逻辑（最简实现）

`POST /api/model-providers/{id}/test`

Service 里直接用 `RestTemplate` 或 `OkHttp` 调用对应提供商的健康检查接口，异常分支：

```
SocketTimeoutException / ConnectTimeoutException
  → throw new BizException(ErrorCode.CONNECT_TIMEOUT)

HTTP 401 / 403
  → throw new BizException(ErrorCode.AUTH_FAILED)

HTTP 404 / 模型列表为空
  → throw new BizException(ErrorCode.MODEL_NOT_FOUND)
```

`GlobalExceptionHandler` 捕获 `BizException`，统一返回 `R.fail(e.getCode(), e.getMessage())`。

六、关键依赖（pom.xml 新增）

```
<!-- MyBatis-Plus -->
<dependency>
    <groupId>com.baomidou</groupId>
    <artifactId>mybatis-plus-boot-starter</artifactId>
    <version>3.5.5</version>
</dependency>

<!-- MySQL -->
<dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
</dependency>

<!-- Lombok（已有则跳过）-->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
</dependency>
```

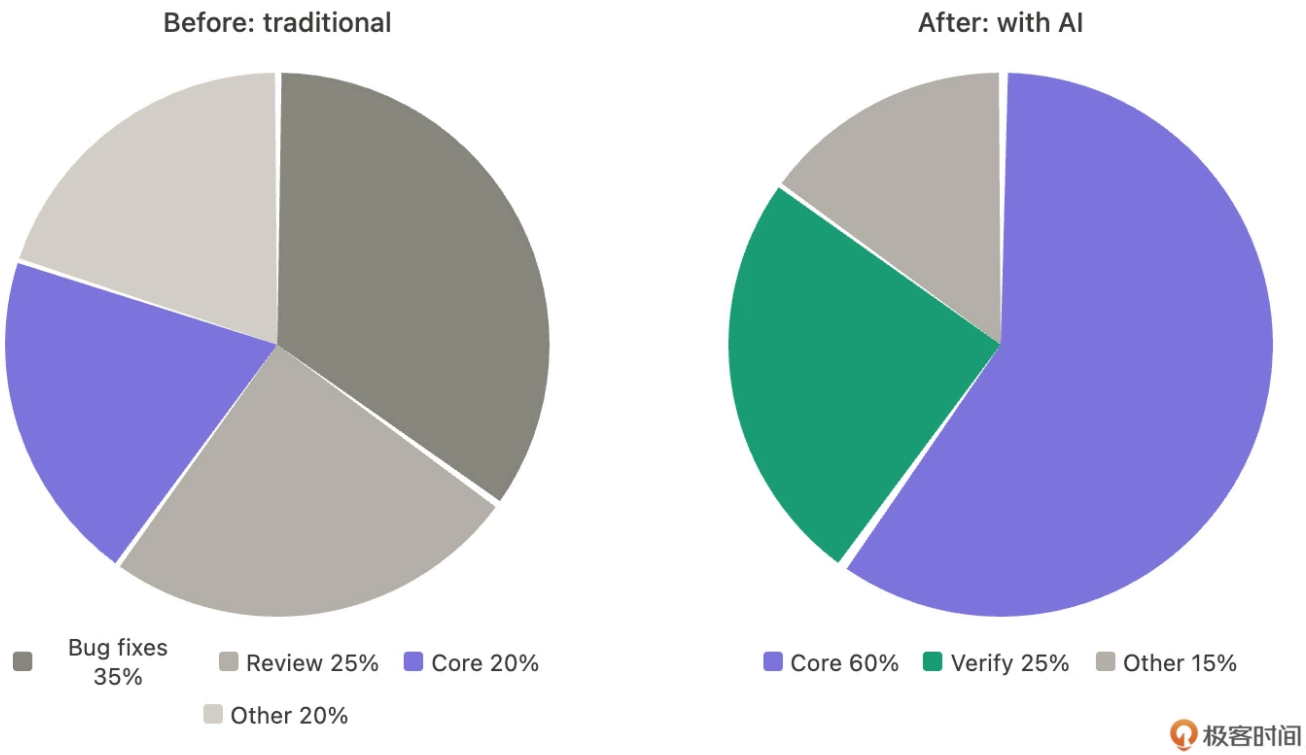
同一个 AI，这两种指令拿到的输出质量完全是两回事。第二种方式的输出结构完整，有错误码、有依赖管理、有连通性测试，是一个成熟项目该有的样子。

建立这个框架之后，你的核心工作不再是“写代码”，而是“做决策”——决定做什么不做什么，决定用什么方式做，决定做到什么标准，验收 Claude Code 做出来的结果。

你可能会问：那我是不是不用懂技术了？恰恰相反。你必须有足够的技术判断力来验证它的输出。看不懂它写的代码，就没法判断对不对；不了解技术细节，就没法给出精确的任务描述。技术能力是你当好“架构师”的前提。

我想多说一点，关于这个框架真正改变的是什么。

很多人讨论 AI 提效，关注的都是“写代码快了多少”。但我体会最深的不是速度变化，而是精力分配的变化。做一个系统，大量时间其实不是花在写新功能上——修低级 bug、反复 review、写测试、写文档，这些又多又碎的活消耗了大部分精力。AI 把这些接过去之后，我可以把大部分精力集中在架构设计和核心决策上。这个改变是复利的。



传统精力分配 vs AI 辅助后

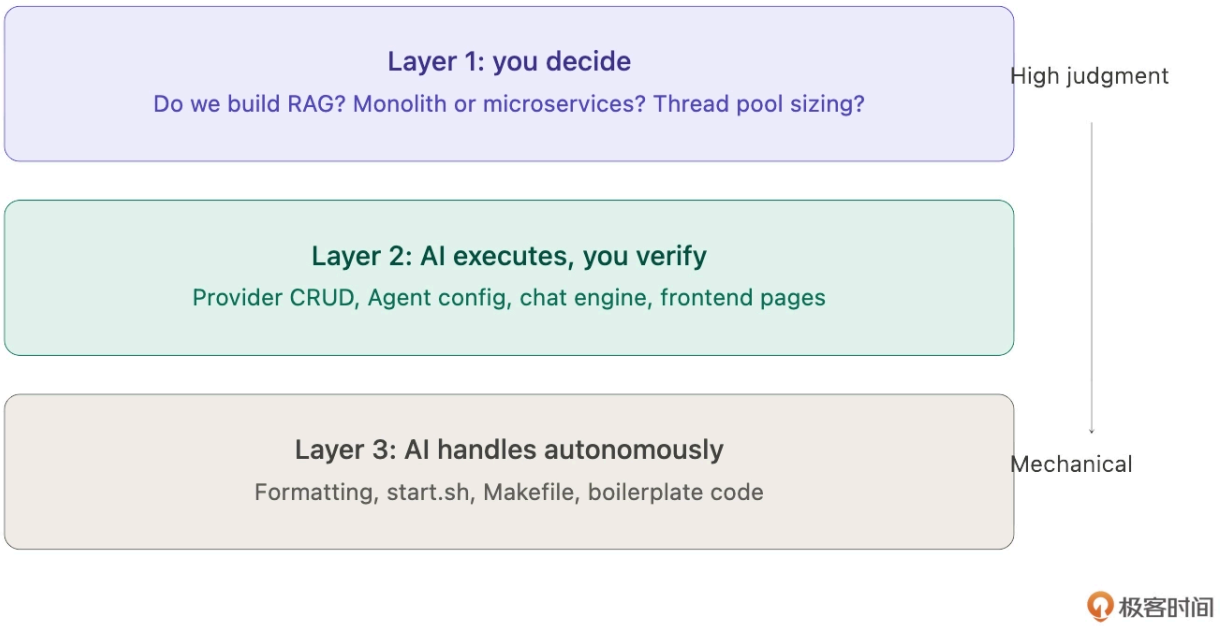
所以“你是架构师，它是工程团队”的深层含义是：AI 让你终于有条件把全部精力放在最该做的事上。

那具体怎么分工？我把它分成三层，你可以直接拿来用。

第一层，必须你做的。产品边界、架构决策、技术取舍、跨模块一致性。拿 Hify 来说：做不做 RAG 知识库？用微服务还是模块化单体？线程池参数设多少？这些决策只有你能做。Claude Code 可以给你方案对比（后面第 04 讲你会看到这个过程），但拍板的必须是你。

第二层，AI 做你验收的。业务代码、接口开发、前端页面、测试用例、文档。这是工作量的大头。有一个硬标准：任何一行代码你都要能说清楚它在干什么。说不清楚，就不能用。

第三层，AI 全权处理的。格式化、样板代码、简单重构、启动脚本、Makefile。扫一眼没问题就行。



从上到下：你做 → AI 做你验收→ AI 全权处理
越往上面越需要判断力

这三层的边界不是死的。做 Hify 这种应用层项目，第二层比例大，AI 可以写更多代码。做基础设施项目（分布式系统、存储引擎），第一层比例大，核心代码最好自己写。关键是你始终清楚哪些东西不能放手。

还有一点要提前说：有时候你还没完全想清楚要做什么，比如“业务开发前需要准备哪些基础组件”这种问题。这时候不是只有“想清楚再让它做”这一条路，你也可以先让 Claude Code

帮你梳理思路，它帮你想，你来判断取舍。这个协作模式我们在后面会专门讲。

怎么检查它给你的东西

分工定了，验收怎么做？我用三步，按优先级来。

我拿 Hify 里一个具体场景来说明。假设你让 Claude Code 实现“删除模型提供商”的接口，你的指令是：“实现删除模型提供商的接口，路径 DELETE /api/v1/providers/{id}，删除前检查是否有 Agent 正在使用该提供商，如果有则拒绝删除。”

Claude Code 的主要输出是：

核心逻辑如下：

```
// ModelProviderController.java
@DeleteMapping("/api/v1/providers/{id}")
public R<Void> delete(@PathVariable Long id) {
    providerService.deleteById(id);
    return R.ok();
}

// ModelProviderService.java
public void deleteById(Long id) {
    // 1. 检查提供商是否存在
    ModelProvider provider = baseMapper.selectById(id);
    if (provider == null) {
        throw new BizException(ErrorCode.NOT_FOUND, "提供商不存在");
    }

    // 2. 检查是否有 Agent 正在使用
    Long usingCount = agentMapper.countByProviderId(id);
    if (usingCount > 0) {
        throw new BizException(ErrorCode.PROVIDER_IN_USE, "该提供商已被 " + usingCount + " 个 Agent 使用，无法删除");
    }

    // 3. 逻辑删除
    baseMapper.deleteById(id);
}

// AgentMapper.java
@Select("SELECT COUNT(*) FROM agent WHERE provider_id = #{providerId} AND deleted = 0")
Long countByProviderId(@Param("providerId") Long providerId);

// ErrorCode.java 新增
PROVIDER_IN_USE(1004, "提供商使用中，无法删除"),
```

Claude Code 很快给了代码。核心思路就三步：查存在 → 查引用 → 删除，BizException 由 GlobalExceptionHandler 统一拦截返回对应错误码。现在我们用三步检查法来看。

第一步，查意图：它做的是不是你让它做的?

你打开代码，发现它确实实现了删除接口和关联检查。但它额外做了两件事：加了一个软删除机制（不是真删，标记 deleted=true），还加了一个操作日志记录。

软删除合不合理？也许合理。操作日志要不要？可能要。但问题是你没要求。它自作主张扩大了范围。这次看着还行，下次它可能就“顺手”加一些不合适的东西。

所以第一步的判断：功能多了，需要砍掉或确认后再保留。

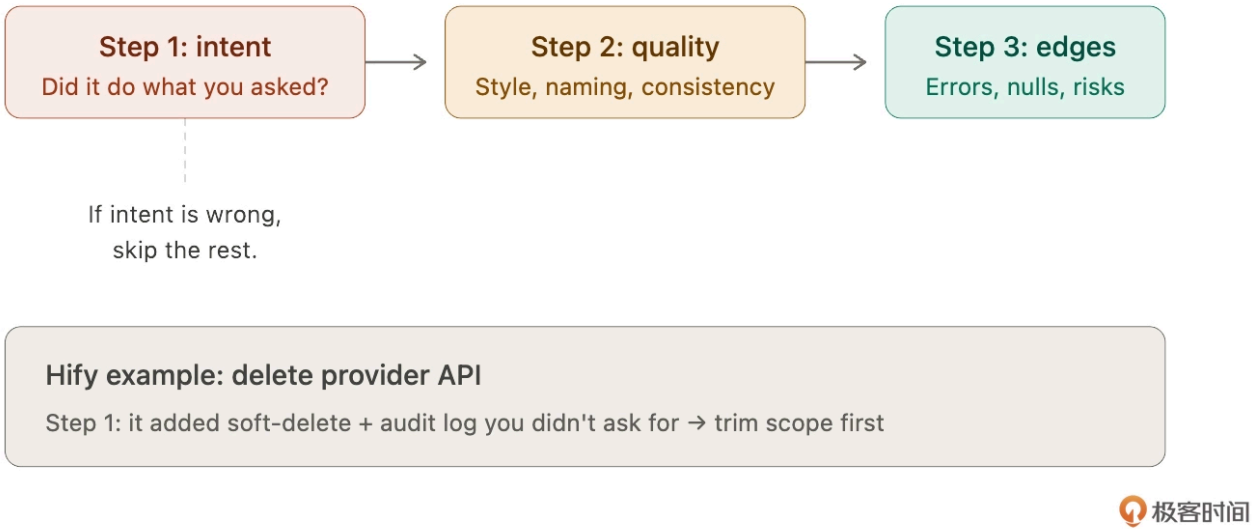
第二步，查质量：风格、规范、一致性。

你看命名，发现检查关联的方法叫 checkProviderUsage，但项目里其他类似方法都叫 validateXxxBeforeDelete。返回格式上，其他接口返回 Result<Void>，它这里直接返回 ResponseEntity。这些不是 bug，功能没问题，但放在整个项目里风格不统一，时间一长，每个模块一种风格，维护起来很痛苦。

第三步，查边界：错误处理、异常、潜在风险。

你发现一个问题：它检查了“有 Agent 在使用则拒绝删除”，但没有处理并发场景——检查通过后、删除前，刚好有新 Agent 绑定了这个提供商怎么办？另外，传入的 ID 不存在时，它抛了 500 而不是 404。

这些边界问题，你可以让 Claude Code 自己再检查一遍——它做细节排查很稳定，不会因为疲劳漏掉。



三步检查法流程图：意图→质量→边界

顺序很重要：先查意图，再查质量，最后查边界。如果意图就错了，后面两步白查。

最后提醒一个坑：Claude Code 连续几次输出都没问题时，你会不自觉放松警惕。这是人的认知规律，不是意志力能对抗的。怎么办？靠机制——关键模块强制跑测试再接收，别光靠肉眼。这个后面测试那一讲会详细说。

总结

这节课我们建立了一个认知框架和两个实操工具。

认知框架：你是架构师，Claude Code 是你的工程团队。你的工作重心从写代码变成做决策。

实操工具一，三层分工：哪些必须你做、哪些 AI 做你验收、哪些 AI 全权处理。从下一讲开始，你每次给 Claude Code 派任务前，先想一秒这个任务在哪一层。

实操工具二，三步检查法：拿到 AI 输出后，先查意图、再查质量、最后查边界。意图错了后面白查，这个顺序不能反。

下一讲我们解决一个具体问题：怎么让 Claude Code 永远在你的轨道上跑，不自由发挥。这就是贯穿整门课的核心方法论——规范驱动开发。

思考题

回想一下你现在用 AI 编码工具的方式。在今天说的三层分工里，有没有哪些本该你拍板的事情，不知不觉交给了 AI ？比如某个技术选型、某个架构决策，你是自己想清楚了再让 AI 执行，还是直接问 AI“你觉得该怎么做”然后就照着做了？

欢迎你在留言区与我交流！如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (24)

最新　精选



winter

2026-04-08 来自安徽

有一个问题，工程师的review效率与CC的代码产出效率是不成比例的，面对海量代码，尤其是复杂业务代码，如何review呢?

> 作者回复: 这个最后的时候会讲。当前简短回答下，先不展开：
> 1. 每个系统都有核心逻辑，核心逻辑研发来把控
> 2. 可以让AI来review AI。

共 4 条评论 >　　8

星辞Sting

2026-04-26 来自浙江

本人也是开发工程师。有时候面对存量的屎山代码，一个很恶心的需求改动。我自己都实在想不好该如何下手才能完全改正确，功能边界和影响面无法完全覆盖，这种情况我如何能给AI清晰的定义？面对这种我自己也没完全想清楚的情况下，AI如何来帮助我。 虽然最好的做法是重构，重新设计范式和模型，但是业务快速扩展实在不允许

作者回复: 说实话，我觉得其实就一句话： 挺难，但是也不是没办法。或者说有办法通过AI降低难度和工作量。这个展开讲太长了，放不下。下门课，我们会讲老项目改造，5月初上线。

共 3 条评论>

4



海阔天空

2026-04-25 来自山东

关键是经验不足的时候只能听AI的建议进行选择，也是边做边涨经验

作者回复: 没有ai的时候，不也是边做边涨经验嘛。 现在还有ai 给我们建议，以前只能去问人，现在的优势比之前大多了。所以，优势在我～

共 2 条评论>

4



风轻扬

2026-05-06 来自北京

目前让AI做的工作都是简单的CRUD，让AI真正开发前，会做三件事。
1、写好开发文档，然后把文档扔给他，让他学习
2、有些内部工具可以复用的，我会找一个参考的方法，让他学习。
3、先让他给出一份开发计划，看看他咋干活。我觉着有问题的地方，给他纠正下，再让他生成开发计划，如此反复，直到我满意，然后就让他开始干活
这三点做下来，基本

作者回复: 其实大部分人的使用都是这样子。或者这是我自己理解的最合理的方式了。我自己基本也都是使用这个流程。所以你在这个课程中我比较少使用一些高级工具，比如spec-kit之类的。
只是落地中，这种流程很看研发的积累和判断力。如果研发的积累和判断力够，就能得到很好的结果。如果不够的话，就需要一些规范和流程来提高质量。
在我看来，比如SDD的标准化工具，就是通过规范和流程来提高产出质量的。

共 2 条评论>

3

Geek_de51de

2026-04-20 来自江苏

如果我想做一个具体事，是不是从工程项目角度出发定义

作者回复: 这里我个人的看法会有点不一样。我会先从产品角度出发去定义这个事情。想清楚做什么。再从工程出发，决定怎么做。否则长期可能会返工，这很麻烦。

3

努力搬砖的阿凯同学
2026-07-07 来自广东

我以前是把我的需求描述给AI让它帮我分析整理，然后我review没问题了就直接让AI生成代码。现在前面的步骤没变，然后AI会先生成一个spec文档，然后我review没问题或者要修改就让AI改，然后直接修改和生成代码，完成后让AI生成测试用例。最后让AI自己检查有没有模糊的地方，以及我有没有什么没有考虑到的盲点。

作者回复: 是的。其实核心的目的和流程一直没变。变得是谁来干写代码这个活而已。相当于工作量左移了。比如把写代码变为了：“给AI让它帮我分析整理，然后我review，AI生成代码”
需求分析、技术设计、写代码，测试。全流程没变。变得是，我们做哪部分工作，AI做哪部分工作。以及我们怎么指导（SDD），约束（Harness）AI 把代码写的符合我们的预期。

共 2 条评论>

2

gaozhanchuan
2026-04-26 来自北京

使用Claude+Superpower了一段时间，个人感觉第一层工作量大约有30%，第二层工作量大约有60～65%，第三层工作量10%以下。从之前开发人员开会讨论需求、架构方案设计、实际代码开发、需求变更、设计调整、代码修改到现在和Claude聊需求、确定设计方案、审核AI代码，整个过程除了不用亲自写代码，实际的工作量并没有降低甚至还有所增加，如何控制AI不跑偏，按自己实际的需求去超预期完成任务，这些不是普通程序员甚至架构师一时半会能快速掌握的，只能在一次次的实践中慢慢总结和摸索

作者回复: 说实话，其实我的体感跟你不太一样。我的体感是，需求调研，选型，编码量这部分工作量几乎压缩到之前的10%到20%。其他的公司流程（开会啊，沟通啊，评审啊）那没办法了。

我觉得你有这个体感的原因就是你说的这两点。很重要：“如何控制AI不跑偏”，“按自己实际的需求去超预期完成任务，这些不是普通程序员甚至架构师一时半会能快速掌握的”。

我觉得当你熟练度足够高，能解决这两个问题，真的会轻松特别多。

共 2 条评论>

2

Demon
2026-06-08 来自浙江

单纯的前端或后端，不深入学习的前提下，可以转全栈吗？如果可以，那么怎么判断AI输出的代码是合格的，它生成的代码有没有多余的？会不会在某个场景下引发性能问题？

作者回复: 就看你怎么定义“全栈”这个词了。两个角度：
1. 做出东西，完成工作的全栈。
2. 成为每个方向的比较资深的，专家。

第一种，肯定可以，AI太强了，写很多相对简单的代码基本不会出错，对于做出东西，完成工作这个程度的事情，基本不会出错。

第二种，也就是你说的：“那么怎么判断AI输出的代码是合格的，它生成的代码有没有多余的？会不会在某个场景下引发性能问题？”，大概率是这种。

这种的话，我觉得转全栈依旧没问题，而且是趋势。进度也会比之前快。关键是怎么用好ai。然后自己也得花时间去学习，研究每个技术栈，不能浅尝辄止。

核心就是你要具备某个技术栈的判断力。而这个判断力可以在AI的辅助下提速养成，但是你要学习，不能当甩手掌柜。

1

Geek_8ae63e
2026-07-07 来自北京

老师，你的md是怎么定义的，我这里定义不好

作者回复: 我一般是写好技术文档后，让AI 基于这些文档去自动生成。基本都是这样子。我用我的习惯和思路写需求文档、技术文档。很多信息都在这两个文档里面，然后让AI 自己去提炼出来。



Jervis
2026-06-23 来自广东

第二步，查质量：风格、规范、一致性。
风格，规范 最后查就好吧

作者回复: 在我看来，顺序应该没那么严格～